

Electronic Companion

Limited-Memory Renewal Contracts: Exact Pooling and Selected-Book Design

EC.1 Scope and Reading Map

This companion provides the prerequisites, reproducible experiment definitions, and complete new execution records for the current article. The main article contains the complete proofs of terminal-first pooling, the group-box identity and explicit cover count, the mandatory-prefix recurrence, the completion-tree theorem, and catalog and threshold robustness. The two antecedent results below document the exact response-type reduction, the already established fixed-budget enumeration implication, and the individual-box convergence argument on which the group-box construction builds.

No historical mathematical source or recorded failed experiment has been deleted. The directory `revisions/or-r51-selected-book-frontier-20260925/predecessor/` contains byte-identical copies of the reviewed main article and mathematical companion. All original revision directories, including continuous-design, institutional, Monge, resource-augmentation, and harmonic-coarsening derivations, remain unchanged. The machine-readable content map identifies current and archival locations. Those archival readers are not additional required appendices to this journal-facing article. References to a retained result explicitly use its preserved label; cross-document links lead to the relevant archived reader when it is not reproduced here.

EC.2 Response Types and the Complexity Map

Write $P_m(B) = \mathcal{V}_{m,\mathcal{A}}^\tau(B; \rho)$ and $R(c) = \sum_{a_i \in c} \rho_i$. All results in this section concern a fixed finite catalog $\mathcal{A} = (a_1 < \dots < a_N)$. A response type consists of a cap, a shortfall cost function, and an eligible catalog prefix. Let d denote the number of such types; this is distinct from the quadratic reward-curvature parameter q in the continuous-design appendix.

Proposition EC.2.1 (Exact response-type reduction). *Partition branches into groups with the same b_j , the same h_j on $[0, b_j]$, and the same set $\mathcal{A} \cap [0, \tau_j]$. Replace each group by one branch whose weight is the sum of its weights and whose ceiling is the minimum of its ceilings. The reduced and original joint catalog problems have the same optimum at every feasible B and every budget m . A reduced solution lifts by assigning its target and canonical lottery to every branch in its group.*

Proof. Every book is a subset of $\mathcal{A}$. Group members therefore have exactly the same eligible book and the same concave response v_c . For a group of total weight w , replace its targets by their weighted average $\bar{t}$. This preserves its contribution to the root equality, keeps the average between the common anchor and cap, and gives $wv_c(\bar{t}) \geq \sum_{j \text{ in group}} \pi_j v_c(t_j)$. Neither the book nor its charge changes. Conversely, the common reduced target and its canonical realization satisfy every original ceiling because the eligible catalog prefix is unchanged. The two feasible mappings prove equality of optimal values. $\square$

The common-ceiling homogeneous aggregation argument already appears in the retained continuous analysis. The present statement uses catalog-prefix equivalence and permits distinct numerical ceilings. Equal caps alone do not suffice when either the cost or eligible prefix differs. Forming rational quadratic signatures by binary searching the ordered catalog takes $O(k \log N)$ prefix comparisons, plus grouping and output work. With hashing, grouping uses expected linear dictionary operations; comparison-based sorting instead adds $O(k \log k)$ comparisons. General cost-function equality requires a supplied exact representation or a verified type partition.

Proposition EC.2.2 (Positive complexity regimes). *For rational quadratic primitives: (i) one response type or $B = \bar{b}$ is solved by one invocation of Theorem 3.1; (ii) for every fixed m , exact joint catalog design is polynomial in N , k , and input bit length by enumerating at most $\sum_{s=1}^m \binom{N}{s}$ books and applying the fixed-book allocator; and (iii) the adaptive method below has polynomial work per price per node and a finite additive scheme in dimension $d - 1$.*

Proof. With one type the root equality fixes its target. At saturation positive weights force every target to its cap. For a specified book the concave allocation and its supporting-price certificate are polynomial for rational quadratic data, so enumerating the stated finite collection proves (ii). Theorems 5.1 and EC.3.1 prove (iii). $\square$

Enumeration is an XP bound in m , not a fixed-parameter bound of the form $F(m)N^c$ with a uniform exponent. A hardness assertion with a fixed small m would contradict (ii) unless P equals NP. We establish neither general NP-hardness nor a polynomial-time algorithm when both m and the number of distinct types vary. The additive scheme below remains exponential in d in its worst case. This map distinguishes proved upper bounds from unresolved lower bounds rather than treating the earlier grid's dimension as an intrinsic impossibility.

EC.3 Adaptive Joint Design and Verifiable Global Intervals

Let $\Theta = \{t \in [a_1, b] : \sum_j \pi_j t_j = B\}$. A box can be reduced without losing any point of its intersection with Θ . With $s_\ell = \sum_j \pi_j \ell_j$ and $s_u = \sum_j \pi_j u_j$, its exact coordinate projections are

$$\ell'_j = \max \left\{ \ell_j, \frac{B - s_u + \pi_j u_j}{\pi_j} \right\}, \quad u'_j = \min \left\{ u_j, \frac{B - s_\ell + \pi_j \ell_j}{\pi_j} \right\}. \quad (\text{EC.1})$$

The box is discarded only if $B \notin [s_\ell, s_u]$ or an interval is empty. Formula (EC.1) follows by allocating all other coordinates to their upper or lower endpoints; continuous interval filling proves attainability of each projected endpoint.

At each retained box compute $U_X = \min_{\lambda \in \Lambda} U_X(\lambda)$ for any finite price set containing zero. Every recovered book has an anchor in I_X and hence has a feasible allocation satisfying the original equality. Optimize that book's targets exactly, either within X or over the entire original target region, and update the feasible incumbent L . The implementation uses global fixed-book allocation and caches repeated books.

This changes neither the original budget nor the charge schedule. Root price bisection is a choice of bound-strengthening prices, not an assertion of zero duality gap or a prerequisite for validity.

Retain the leaves of a binary cover of Θ , select a leaf with largest upper bound, and bisect one independent target coordinate at its midpoint. Apply (EC.1) to both children. Their intersections with Θ cover their parent's intersection; the common boundary may overlap. A dependent coordinate j_0 is recovered through the equality. Use as splitting coordinate the longest weighted side $\pi_j(u_j - \ell_j)$ among $j \neq j_0$. The code chooses a largest-weight coordinate as j_0 ; this choice is not needed for validity.

Theorem EC.3.1 (Adaptive original-budget certificate and finite accuracy). *Assume f and all h_j are Lipschitz on their relevant compact intervals and put $L_v = \max\{L_f, \max_j L_{h,j}\}$. Use the exact oracle in Theorem 5.1, include price zero at every node, and evaluate the returned zero-price book by a feasible exact fixed-book allocation. At every interruption,*

$$L \leq P_m(B) \leq U = \max\{L, \max_{X \text{ a leaf}} U_X\}. \quad (\text{EC.2})$$

For every $\varepsilon > 0$, best-upper-bound selection and longest-weighted-side bisection terminate with $U - L \leq \varepsilon$. If $d = 1$ the root is a singleton and is exact. For fixed $d > 1$, a conservative full-tree bound is $2^{H+1} - 1$ nodes, where

$$H = \sum_{j \neq j_0} \max \left\{ 0, \left\lceil \log_2 \frac{2L_v(d-1)w_j^0}{\varepsilon} \right\rceil \right\}, \quad w_j^0 = \pi_j(u_j^0 - \ell_j^0). \quad (\text{EC.3})$$

A zero width contributes zero. With $L_v = 0$, one root evaluation suffices. The bound is polynomial in numerical $1/\varepsilon$ for fixed d , and exponential in d in general; adaptive pruning can finish before this worst-case cover is constructed.

Proof. A feasible policy supplies the lower bound. Weak duality and the complete leaf cover supply the upper bound, irrespective of the search order or the quality of the incumbent. Thus pruning is valid whenever a leaf bound is at most $L + \varepsilon$, and retaining such leaves in a certificate is harmless.

For local tightness, let c^0 maximize $S_X(0)$ and let $\hat{t}$ be its independently maximizing box targets. Condition (18) gives some equality-feasible t^0 for that same book in X . The canonical response is continuous and piecewise concave with slopes bounded in absolute value by L_v : its chord slopes are bounded by L_f and its shortfall slopes by $L_{h,j}$. Charges cancel for the same book. Therefore

$$U_X \leq U_X(0) \leq \sum_j \pi_j v_{j,c^0}(t_j^0) - R(c^0) + L_v \sum_j \pi_j(u_j - \ell_j) \leq L + L_v \sum_j \pi_j(u_j - \ell_j). \quad (\text{EC.4})$$

An exact global fixed-book allocation can only improve the displayed feasible value. The inequality does not require a uniform bound on a recovery-compression penalty and does not mix two books.

After coordinate projection, the dependent weighted width is no larger than the sum of the other weighted widths. To see this, choose equality-feasible points attaining its lower and upper endpoints and subtract their equalities. Thus the last sum in (EC.4) is at most twice the sum of the independent weighted widths. No node needs further splitting once each independent width is at most $\varepsilon/[2L_v(d-1)]$. Before this occurs, a longest-width split halves a width above that threshold; projection cannot increase any width. Along a path,

coordinate j can be split at most its summand in (EC.3) before falling below the threshold. Hence every path needing refinement has at most H splits. The complete binary tree bound follows. At a singleton box the same calculation gives equality of the box upper bound and the fixed-target optimum. This also proves the one-type case. $\square$

The result replaces completion of a full target net by certificate-directed refinement. It does not promise that refinement is inexpensive for every input. It supplies a structural polynomial node oracle and a meaningful interval when it is not. An absolute tolerance is measured in payoff units. Our quadratic experiments normalize target levels to $[0, 1]$ and use $r = 2, q = 1$; they report both the absolute gap and its ratio to $r + \max_j \gamma_j/2 + \sum_{i=1}^m \rho_{(i)}$, where the charges are sorted decreasingly. This positive declared scale remains meaningful when net value is close to zero. Rescaling all rewards and charges requires rescaling the requested tolerance.

EC.3.1 Independent checking and physical implementation

A certificate contains the original inputs, the exact type partition, an original-space lottery, every binary split, the authoritative leaves, and one upper-bounding price table per node. It need not contain every price tried. The checker verifies anchor bounds, every reachable forward Bellman inequality, every backward stopping and transition inequality, and the final maximum in (28). Upper tables need not equal the optimizer's tables; satisfying these inequalities is enough to dominate every represented path. It separately reconstructs both projected children at every split and verifies that no leaf is missing. Finally, it checks probabilities, selected charges, pre-draw service, the exact root promise, and supportwise realization limits without using the canonical response formula. The certificate therefore certifies all target regions, not just the sampled incumbents. Its storage is $O(TmN + Td)$ rational entries for T recorded nodes, apart from the policy and input data; the checker recomputes edge values and has polynomial work per recorded node.

The terminal alphabet is only one physical resource. A type-reduced implementation additionally stores a history-to-type encoder, a target and lottery table, command constants, and probability representations. Arbitrary rational probabilities may require variable-length random-bit generation or a separate finite-precision error analysis; we do not count them as free writable memory. Offline certificates have the size just stated and are not assumed to fit in the terminal gateway. The total deployment cost is consequently not identified with $\lceil \log_2 m \rceil$ bits. This separation is essential to interpreting the repeated-type computation as joint design of a restricted interface, rather than a claim about full-system memory or calibrated field performance.

EC.4 Frozen Experiment Definitions

The immutable protocol is `PROTOCOL.json` in the current revision directory. It was frozen before the local run, with SHA-256

095b295a4dfc865b4164c96eb69b3f801f773b496716b5ee06a07a90c44f3cc2.

It is not described as a remotely preregistered protocol. Local execution and any publication rerun have separately identified environment records. The inputs are exact rational model primitives; displayed decimals

are generated only after verification. Runtime measurements belong to the environment that produced each record, not to an earlier revision’s machine.

The stress population uses exactly the six seeds 49500–49505 from the preceding stress fixture, with 12 histories, nine candidates, and budget three. The node ladder is (1, 7, 31, 127), the local price-trial ladder is (0, 2, 5), and each timed request has a three-second allowance. Each of the six completion requests uses at most 10,000 nodes and a 30-second allowance. These produce 72 interruption requests and six completion requests without selecting new favorable instances.

The strict-prefix catalog is $a_i = (i/(N - 1))^2$ for $i = 0, \dots, N - 1$. Its common eligible index is $\lfloor 2(N - 1)/3 \rfloor$. Distinct ceilings are placed strictly inside the following catalog gap, and heterogeneous caps remain below the smallest ceiling; some may exceed the last eligible catalog level. We use $k \in \{16, 64, 256, 1024\}$, $N \in \{9, 17\}$, budget three, and promise fractions $1/3$ and $9/10$ of the mean cap. Four additional cases use $N = 33$, $k \in \{16, 64\}$, budgets two and four, and promise fraction $9/10$. Every case uses the exact one-class pooled oracle; it is not counted as a new completion-tree timing.

The matched population has 16 histories, 17 candidates, budget three, and fixed costs, weights, caps, charges, and promise. Only ceilings change to produce $E = 1, \dots, 8$. Node allowances are 1, 15, and 127, and price-trial allowances are zero and three. The 48 resulting requests therefore isolate this eligibility change rather than comparing different history or catalog scales.

The refinement population begins at $\{0, 1/4, 3/4, 1\}$ and inserts $1/2, 23/50, 27/50, 12/25$, and $13/25$ in that order. Each of six stages is evaluated with inserted-level charge three and $1/200$, with screening both off and on, for 24 requests. The eight histories, their primitive caps and costs, and their promise do not change. The boundary population uses eight histories and catalog $\{0, 1/4, 1/2, 3/4, 1\}$; it moves zero, one, four, or eight ceilings above $1/2$, using perturbations $1/100, 1/10000$, and $1/1000000$. All twelve requests remain in the population.

EC.5 Checker and Execution Instructions

The selected-prefix checker imports only the Python standard library. It does not call the optimizer, an optimization solver, or the optimizer’s support routine. It reconstructs each candidate union from the include/exclude decisions; verifies the anchor and mandatory levels; evaluates scalar supports at knots, bounds, and stationary points; and verifies the supplied forward and backward upper potentials. At every branching node it requires both include and exclude children. Every pruned or open leaf needs its own upper witness, and the final endpoint must dominate all surviving leaves. It independently evaluates the final policy in the uneliminated model, including every positive-probability support point and every opening charge. The experiment driver also verifies that each certificate’s model equals its declared instance.

Compressed bytes mean deterministic gzip bytes for the saved certificate in that execution. Uncompressed bytes, counts of rational entries, maximum numerator and denominator bit lengths, checker times, and checker-to-search time ratios are all computed from the saved objects. Changing the problem requires revalidation; a winning book’s smaller partition is not a substitute for coverage of competing books. Numerical mixed-integer solutions have a different evidence status and are never passed off as exact certificates.

From the repository root, the following commands reproduce the current experiment and readers. To measure fresh timings, use a separate clean checkout and keep the protocol unchanged; do not overwrite the committed record in the archival checkout.

```
R=revisions/or-r51-selected-book-frontier-20260925
python "$R/code/tests.py"
python "$R/code/study.py"
python "$R/code/additional_checks.py"
python "$R/code/tables.py"
python "$R/code/build.py"
```

A single selected-prefix certificate can be checked independently with

```
python "$R/code/check_completion.py" path/to/certificate.json.gz.
```

The publication workflow pins the Python and scientific-library versions and records the actual environment. The old pooling suite is executed with its output directory redirected into this revision, so inherited evidence remains untouched.

EC.6 Capacity-Reservation Calculation

The demand-derived model in the main article is evaluated on three site states with probabilities $(1/4, 1/2, 1/4)$, caps $(3/10, 11/20, 4/5)$, engineering ceilings $(2/5, 13/20, 19/20)$, and backup curvatures $(0, 2, 7)$. The certified batch catalog is $\{0, 1/5, 2/5, 3/5, 4/5, 1\}$, budget three, and the opening charge of index $i = 0, \dots, 5$ is $i/100$. The contract promises nine tenths of the weighted cap. Uniform independent demand and unit revenue two imply $f(x) = 2x - x^2$, so these data have $(r, q) = (2, 2)$.

The exact optimum is $67/100$. The saved policy, selected book, supporting bounds, and original-space verification appear in `results/application.json`. For each used batch x , the check computes

$$2 \int_0^1 \min\{u, x\} du = 2 \left(\int_0^x u du + \int_x^1 x du \right) = 2x - x^2$$

using exact rational arithmetic, then averages over the implemented lotteries and subtracts backup and certification costs. This agrees with the canonical response, exhaustive fixed-book reference, and selected-prefix certificate. It is an independently formulated demand integral, not a fitted or observed application outcome.

EC.7 Complete New Execution Population

Every row below is an executed request, including all interrupted requests. Row number is one-based index in `results/study.json`; its full identifier and input, node allowance, price-trial allowance, solver, rational endpoints, and certificate digest are stored in that same record. The letter E means exact rational width zero; O means open. Rounded display values do not determine status. Relative gaps use the explicitly declared denominator $\max\{|L|, |U|, 10^{-6}\}$.

Table EC.1: All 182 requests and their certificate costs

Row	Family	E	Nodes	Status	Lower	Gap	Relative %	Seconds	Bytes
1	stress	6	1	O	0.30208	0.106	26	0.011	644
2	stress	6	7	O	0.30208	0.0136	4.32	0.018	746
3	stress	6	11	E	0.30208	0	0	0.020	791
4	stress	6	11	E	0.30208	0	0	0.027	794
5	stress	6	1	O	0.30208	0.00573	1.86	0.018	733
6	stress	6	7	O	0.30208	0.0136	4.32	0.049	859
7	stress	6	11	E	0.30208	0	0	0.054	906
8	stress	6	11	E	0.30208	0	0	0.053	906
9	stress	6	1	O	0.30208	0.00426	1.39	0.029	727
10	stress	6	7	O	0.30208	0.0136	4.32	0.097	852
11	stress	6	11	E	0.30208	0	0	0.101	897
12	stress	6	11	E	0.30208	0	0	0.100	900
13	closure	6	11	E	0.30208	0	0	0.053	909
14	stress	7	1	O	0.52947	0.0651	11	0.011	789
15	stress	7	7	O	0.52947	0.0642	10.8	0.017	896
16	stress	7	25	E	0.53564	0	0	0.036	983
17	stress	7	25	E	0.53564	0	0	0.035	984
18	stress	7	1	O	0.52947	0.0517	8.9	0.019	861
19	stress	7	7	O	0.53564	0.02	3.6	0.054	942
20	stress	7	21	E	0.53564	0	0	0.098	1144
21	stress	7	21	E	0.53564	0	0	0.097	1146
22	stress	7	1	O	0.53564	0.0359	6.28	0.031	812
23	stress	7	7	O	0.53564	0.02	3.6	0.107	1022
24	stress	7	19	E	0.53564	0	0	0.186	1250
25	stress	7	19	E	0.53564	0	0	0.184	1252
26	closure	7	21	E	0.53564	0	0	0.096	1145
27	stress	6	1	O	0.45919	0.04	8.01	0.011	661
28	stress	6	3	E	0.45919	0	0	0.012	716
29	stress	6	3	E	0.45919	0	0	0.012	715
30	stress	6	3	E	0.45919	0	0	0.012	717
31	stress	6	1	E	0.45919	0	0	0.013	712
32	stress	6	1	E	0.45919	0	0	0.014	714
33	stress	6	1	E	0.45919	0	0	0.013	711
34	stress	6	1	E	0.45919	0	0	0.014	714
35	stress	6	1	E	0.45919	0	0	0.013	712
36	stress	6	1	E	0.45919	0	0	0.013	713
37	stress	6	1	E	0.45919	0	0	0.013	711
38	stress	6	1	E	0.45919	0	0	0.013	714
39	closure	6	1	E	0.45919	0	0	0.013	716
40	stress	8	1	O	0.030152	0.0664	68.8	0.010	627

Row	Family	E	Nodes	Status	Lower	Gap	Relative %	Seconds	Bytes
41	stress	8	7	E	0.030152	0	0	0.015	722
42	stress	8	7	E	0.030152	0	0	0.015	722
43	stress	8	7	E	0.030152	0	0	0.016	724
44	stress	8	1	O	0.030152	0.00644	17.6	0.015	670
45	stress	8	7	E	0.030152	0	0	0.032	769
46	stress	8	7	E	0.030152	0	0	0.031	769
47	stress	8	7	E	0.030152	0	0	0.031	772
48	stress	8	1	O	0.030152	0.00644	17.6	0.024	670
49	stress	8	7	E	0.030152	0	0	0.055	771
50	stress	8	7	E	0.030152	0	0	0.054	771
51	stress	8	7	E	0.030152	0	0	0.055	772
52	closure	8	7	E	0.030152	0	0	0.031	774
53	stress	5	1	E	0.46425	0	0	0.012	671
54	stress	5	1	E	0.46425	0	0	0.012	673
55	stress	5	1	E	0.46425	0	0	0.011	673
56	stress	5	1	E	0.46425	0	0	0.011	674
57	stress	5	1	E	0.46425	0	0	0.012	671
58	stress	5	1	E	0.46425	0	0	0.012	675
59	stress	5	1	E	0.46425	0	0	0.011	673
60	stress	5	1	E	0.46425	0	0	0.011	676
61	stress	5	1	E	0.46425	0	0	0.011	673
62	stress	5	1	E	0.46425	0	0	0.011	675
63	stress	5	1	E	0.46425	0	0	0.012	674
64	stress	5	1	E	0.46425	0	0	0.012	675
65	closure	5	1	E	0.46425	0	0	0.012	677
66	stress	6	1	O	0.43676	0.101	18.8	0.011	655
67	stress	6	7	O	0.43676	0.0368	7.77	0.017	757
68	stress	6	11	E	0.43676	0	0	0.022	798
69	stress	6	11	E	0.43676	0	0	0.022	799
70	stress	6	1	O	0.43676	0.00363	0.825	0.019	732
71	stress	6	7	O	0.43676	0.0368	7.77	0.047	867
72	stress	6	11	E	0.43676	0	0	0.059	910
73	stress	6	11	E	0.43676	0	0	0.057	911
74	stress	6	1	O	0.43676	0.00363	0.825	0.031	730
75	stress	6	7	O	0.43676	0.0271	5.85	0.091	939
76	stress	6	11	E	0.43676	0	0	0.109	980
77	stress	6	11	E	0.43676	0	0	0.111	983
78	closure	6	11	E	0.43676	0	0	0.057	914
79	strict	1	1	E	0.13962	0	0	0.017	1167
80	strict	1	1	E	0.37848	0	0	0.021	1512
81	strict	1	1	E	0.13217	0	0	0.041	1515
82	strict	1	1	E	0.38953	0	0	0.052	2249
83	strict	1	1	E	0.073047	0	0	0.060	1944

Row	Family	E	Nodes	Status	Lower	Gap	Relative %	Seconds	Bytes
84	strict	1	1	E	0.29919	0	0	0.077	3207
85	strict	1	1	E	0.1047	0	0	0.143	2528
86	strict	1	1	E	0.35081	0	0	0.186	4647
87	strict	1	1	E	0.1341	0	0	0.234	4679
88	strict	1	1	E	0.38483	0	0	0.300	9768
89	strict	1	1	E	0.15703	0	0	0.553	6129
90	strict	1	1	E	0.39998	0	0	0.731	14421
91	strict	1	1	E	0.1293	0	0	0.967	15614
92	strict	1	1	E	0.38625	0	0	1.204	34761
93	strict	1	1	E	0.14927	0	0	2.255	20137
94	strict	1	1	E	0.36773	0	0	2.894	54454
95	strict	1	1	E	0.42134	0	0	0.154	3333
96	strict	1	1	E	0.42134	0	0	0.161	3665
97	strict	1	1	E	0.40748	0	0	0.553	7163
98	strict	1	1	E	0.41244	0	0	0.578	7490
99	matched	1	1	O	0.61836	0.0138	2.19	0.027	830
100	matched	1	1	O	0.61836	0.00232	0.373	0.043	972
101	matched	1	11	E	0.61836	0	0	0.041	1017
102	matched	1	9	E	0.61836	0	0	0.099	1197
103	matched	1	11	E	0.61836	0	0	0.041	1019
104	matched	1	9	E	0.61836	0	0	0.100	1196
105	matched	2	1	O	0.58585	0.0537	8.4	0.028	796
106	matched	2	1	O	0.61629	0.00439	0.707	0.067	1039
107	matched	2	15	O	0.58585	0.0422	6.72	0.049	1026
108	matched	2	15	O	0.61629	0.00207	0.335	0.314	1615
109	matched	2	41	E	0.61836	0	0	0.086	1365
110	matched	2	25	E	0.61836	0	0	0.373	1730
111	matched	3	1	O	0.61836	0.0138	2.19	0.027	835
112	matched	3	1	O	0.61836	0.00232	0.373	0.046	976
113	matched	3	11	E	0.61836	0	0	0.041	1021
114	matched	3	9	E	0.61836	0	0	0.108	1195
115	matched	3	11	E	0.61836	0	0	0.041	1023
116	matched	3	9	E	0.61836	0	0	0.109	1196
117	matched	4	1	O	0.57665	0.0629	9.84	0.028	802
118	matched	4	1	O	0.60625	0.0104	1.68	0.070	1044
119	matched	4	15	O	0.57665	0.0514	8.19	0.048	1042
120	matched	4	15	O	0.60916	0.000754	0.124	0.339	1826
121	matched	4	45	E	0.60916	0	0	0.091	1478
122	matched	4	23	E	0.60916	0	0	0.392	1970
123	matched	5	1	O	0.58244	0.0571	8.93	0.028	842
124	matched	5	1	O	0.6099	0.00874	1.41	0.072	1038
125	matched	5	15	O	0.58244	0.0456	7.27	0.050	1087
126	matched	5	15	O	0.61251	0.000776	0.127	0.317	1720

Row	Family	E	Nodes	Status	Lower	Gap	Relative %	Seconds	Bytes
127	matched	5	39	E	0.61251	0	0	0.086	1371
128	matched	5	23	E	0.61251	0	0	0.360	1841
129	matched	6	1	O	0.57251	0.067	10.5	0.028	804
130	matched	6	1	O	0.61585	0.00449	0.723	0.072	1050
131	matched	6	15	O	0.57251	0.0552	8.79	0.049	1046
132	matched	6	5	E	0.61585	0	0	0.155	1351
133	matched	6	37	E	0.61585	0	0	0.079	1379
134	matched	6	5	E	0.61585	0	0	0.154	1348
135	matched	7	1	O	0.57909	0.0605	9.45	0.028	848
136	matched	7	1	O	0.60625	0.0104	1.68	0.071	1051
137	matched	7	15	O	0.57909	0.0486	7.74	0.049	1097
138	matched	7	15	O	0.60916	0.000336	0.0551	0.326	1790
139	matched	7	43	E	0.60916	0	0	0.088	1481
140	matched	7	23	E	0.60916	0	0	0.375	1940
141	matched	8	1	O	0.60327	0.0335	5.26	0.029	826
142	matched	8	1	O	0.60327	0.00375	0.617	0.059	1093
143	matched	8	15	O	0.60327	0.00934	1.53	0.048	1078
144	matched	8	15	E	0.60327	0	0	0.228	1635
145	matched	8	29	E	0.60327	0	0	0.068	1287
146	matched	8	15	E	0.60327	0	0	0.224	1639
147	refinement	1	1	E	0.44077	0	0	0.006	731
148	refinement	1	1	E	0.44077	0	0	0.006	728
149	refinement	2	7	E	0.49315	0	0	0.019	795
150	refinement	2	7	E	0.49315	0	0	0.020	794
151	refinement	2	9	E	0.49385	0	0	0.027	845
152	refinement	2	9	E	0.49385	0	0	0.027	845
153	refinement	2	9	E	0.49385	0	0	0.029	857
154	refinement	2	9	E	0.49385	0	0	0.029	856
155	refinement	3	11	E	0.49385	0	0	0.037	890
156	refinement	3	11	E	0.49385	0	0	0.036	891
157	refinement	4	13	E	0.49385	0	0	0.045	926
158	refinement	4	13	E	0.49385	0	0	0.045	925
159	refinement	1	1	E	0.44077	0	0	0.006	731
160	refinement	1	1	E	0.44077	0	0	0.006	730
161	refinement	2	7	E	0.44077	0	0	0.019	881
162	refinement	2	1	E	0.44077	0	0	0.006	740
163	refinement	2	9	E	0.44077	0	0	0.029	1010
164	refinement	2	1	E	0.44077	0	0	0.007	749
165	refinement	2	9	E	0.44077	0	0	0.032	1044
166	refinement	2	1	E	0.44077	0	0	0.008	756
167	refinement	3	11	E	0.44077	0	0	0.040	1129
168	refinement	3	1	E	0.44077	0	0	0.009	759
169	refinement	4	13	E	0.44077	0	0	0.048	1231

Row	Family	E	Nodes	Status	Lower	Gap	Relative %	Seconds	Bytes
170	refinement	4	1	E	0.44077	0	0	0.010	763
171	boundary	1	1	E	0.44077	0	0	0.007	717
172	boundary	2	1	E	0.44077	0	0	0.007	728
173	boundary	2	1	E	0.4457	0	0	0.007	735
174	boundary	1	7	E	0.48815	0	0	0.019	768
175	boundary	1	1	E	0.44077	0	0	0.007	722
176	boundary	2	1	E	0.44077	0	0	0.007	731
177	boundary	2	1	E	0.4457	0	0	0.007	739
178	boundary	1	7	E	0.48815	0	0	0.020	769
179	boundary	1	1	E	0.44077	0	0	0.007	723
180	boundary	2	1	E	0.44077	0	0	0.007	734
181	boundary	2	1	E	0.4457	0	0	0.007	744
182	boundary	1	7	E	0.48815	0	0	0.019	774

EC.8 Both Direct Mixed-Integer Envelopes

The solver primal objective below belongs to the envelope formulation; the actual-policy objective and exact reference are also retained in the machine-readable record. The tangent dual is a numerical upper bound on the original maximization problem. A secant-model dual alone is not an original upper bound without its envelope allowance. Brackets reported in the study use the tangent upper and a numerical original-feasible policy. No floating-point interval is relabeled as a rational proof.

Table EC.2: Complete envelope diagnostics

Case	Envelope	Nodes	Wall (s)	Primal	Dual upper	Solver gap	Error
S49500	tangent	1	0.682	0.3020833	0.3020833	1.8e-16	0.00011
S49500	secant	1	0.020	0.3020833	0.3020833	0	0.00011
S49501	tangent	1	0.018	0.5356402	0.5356402	0	0.00013
S49501	secant	1	0.017	0.5356402	0.5356402	0	0.00013
S49502	tangent	1	0.009	0.4591875	0.4591875	0	4.3e-05
S49502	secant	1	0.010	0.4591875	0.4591875	0	4.3e-05
S49503	tangent	1	0.023	0.03015152	0.03015152	0	5.9e-05
S49503	secant	1	0.015	0.03015152	0.03015152	0	5.9e-05
S49504	tangent	1	0.009	0.4642548	0.4642548	1.2e-16	2.8e-05
S49504	secant	1	0.009	0.4642548	0.4642548	1.2e-16	2.8e-05
S49505	tangent	1	0.012	0.4367578	0.4367578	0	8.8e-05
S49505	secant	1	0.012	0.4367578	0.4367578	0	8.8e-05
E1	tangent	1	0.057	0.6183621	0.6183621	0	3e-05
E1	secant	1	0.041	0.6183621	0.6183621	0	3e-05
E4	tangent	1	0.070	0.6091582	0.6091582	0	3e-05
E4	secant	1	0.083	0.6091582	0.6091582	0	3e-05

Case	Envelope	Nodes	Wall (s)	Primal	Dual upper	Solver gap	Error
E8	tangent	1	0.045	0.6032733	0.6032733	0	3e-05
E8	secant	1	0.043	0.6032733	0.6032733	0	3e-05

References

- Handler GY, Zang I (1980) A dual algorithm for the constrained shortest path problem. *Networks* 10(4):293–309. doi:10.1002/net.3230100403.
- Lawler EL, Wood DE (1966) Branch-and-bound methods: A survey. *Operations Research* 14(4):699–719. doi:10.1287/opre.14.4.699.
- Nielsen F, Nock R (2014) Optimal interval clustering: Application to Bregman clustering and statistical mixture learning. Author manuscript, arXiv:1403.2485.
- Pagès G, Wilbertz B (2012) Intrinsic stationarity for vector quantization: Foundation of dual quantization. Author manuscript, arXiv:1010.4642v2, March 27. Original preprint 2010.
- Patriksson M, Strömberg C (2015) Algorithms for the continuous nonlinear resource allocation problem—new implementations and numerical studies. *European Journal of Operational Research* 243(3):703–722. doi:10.1016/j.ejor.2015.01.029.
- Schoot Uiterkamp MHH, Gerards MET, Hurink JL (2022) Quadratic nonseparable resource allocation problems with generalized bound constraints. *INFORMS Journal on Optimization* 4(2):215–247. doi:10.1287/ijoo.2021.0065.
- Wu X (1991) Optimal quantization by matrix searching. *Journal of Algorithms* 12(4):663–673. doi:10.1016/0196-6774(91)90039-2.